

Singhal heuristic Algorithm - Token Based

→ objective - To remove disadvantage of Suzuki kamei Algo. - msg overhead was high, N - msg to request the token

P_1 → the site request the token
from only those site which have
the probability to have the token.
 P_4 P_2
 P_3 → thus reduce the no of msg
required to execute the CS.

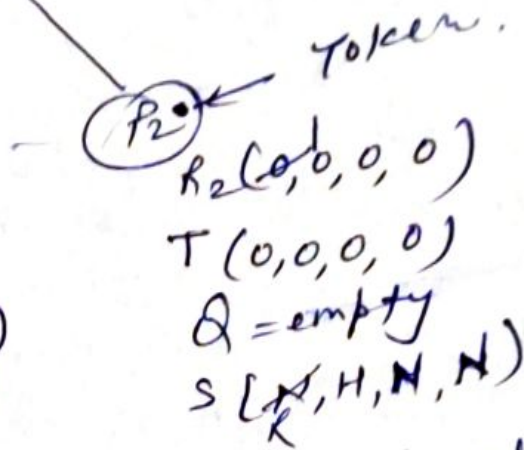
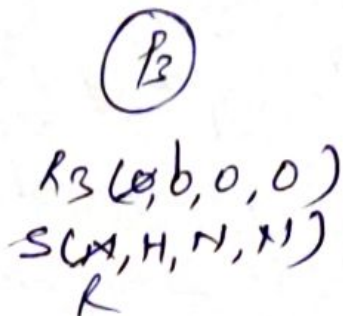
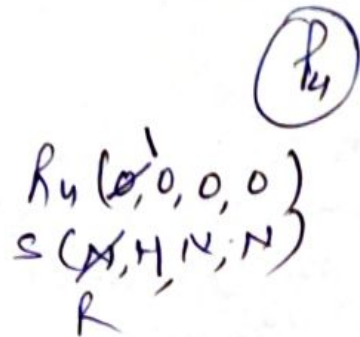
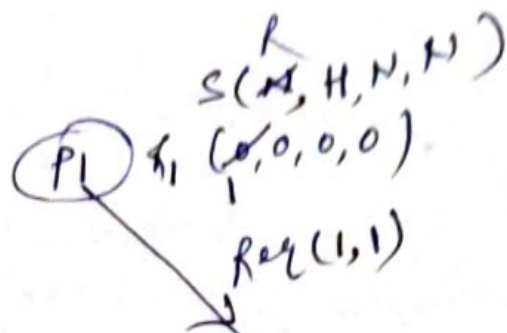
→ How site know this - it uses heuristic - knowledge of state.

A site can be in one of the following states:

- E - executing & Having token
- H - Holding token & not executing.
- R - Requesting the token
- N - Neutral, none of the above.

→ using these state the process will send the request of token to only those site who have highest probability to have the token.

→ the process will send the request who has the state - E, H, R

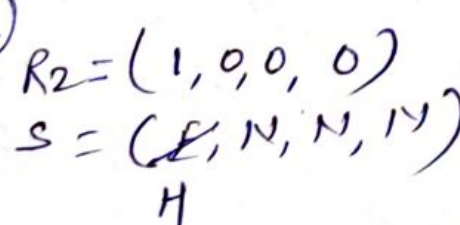
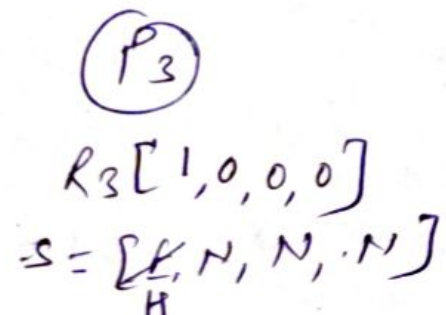
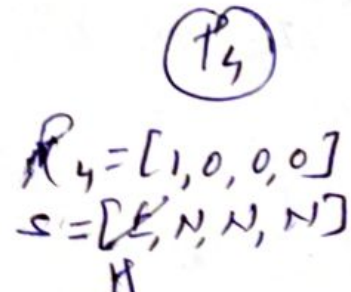
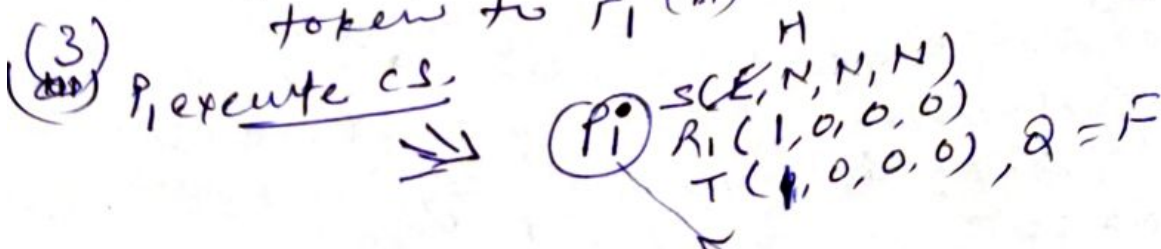


① P_1 requesting cs from P_2 as its state is H.

② P_2 check updated request of P_1 - correct req.

(i) $R_2[1] =$ ~~token array~~
 - update request array of each site

(ii) $T+1 = R$ condition true so P_2 will give token to P_1 (iii) update token array



④ when P_1 release cs. then the states will change. (H,N,N,N)

④ Now P_4 want to execute
 P_4 will send req to P_1
 only bcoz it
 has only
 H state. $req(4,1)$

$S(H, N, N, R)$
 $R = [1, 0, 0, 1]$
 $T = [1, 0, 0, 0]$
 $Q = E$

cs. so if state
 will change
 and request is not
 updated so
 request array is
 also updated.

P_4

$R_4 = [1, 0, 0, 1]$
 $S(H, N, N, R)$

P_2 $R_2 = [1, 0, 0, 1]$
 $S = (H, N, N, R)$

P_3

$R_3 = [1, 0, 0, 1]$
 $S = (H, N, N, R)$

⑤ P_4 will get the token.
 Now P_3 also want to execute cs so it will
 send the request to P_1 and P_4 as both
 states are H & R. states will be updated.

→

P_1 $R = [1, 0, 1, 1]$
 $S = [N, N, R, H]$

P_4

Req.

P_3

P_2 $R_2 (1, 0, 1, 1)$
 $S (N, N, R, H)$

$R_4 = [1, 0, 1, 1]$
 $S = (N, N, R, H)$
 $T = [1, 0, 0, 0]$
 $Q = \{P_3\}$

$R_3 (1, 0, 1, 1)$
 $S = (N, N, R, H)$

⑥ Now P_4 will release cs. and check its
 queue it is non-empty so it will send
 token to P_3 by changing its state.

④ When P_3 executing its cs change then the
 Pj will be:-

$$(P_1) R_1 (1, 0, 1, 1) \\ S = (NN \cancel{EN})_H$$

(P_4)

$$R_4 (1, 0, 1, 1) \\ S = (NN \cancel{EN})_H$$

(P_2)

$$R_2 (1, 0, 1, 1) \\ S = (NN \cancel{EN})_H$$

(P_3)

$$R_3 (1, 0, 1, 1) \\ S = (NN \cancel{EN})_H \\ T = (1, 0, 0, 1) \\ Q = \text{empty.}$$

⑤ Lastly P_3 release the cs. and update
 state and Token array.

Performance :-

Required msg to get the token is

avg - $N/2$, worst = N

Adv - Less msg overhead as compared with
 Suzuki-Kasami

Drawback - Hard to implement.